

Exercice 1

Dérouler le tokenizer de la librairie *SpaCy* étape par étape afin d'extraire la liste de tous les tokens du texte suivant :

Wow! The flight at 6 a.m. wasn't that early.

Exercice 2

1. Soit le **corpus** suivant : « *used use user used code* ». Dérouler le tokenizer Byte-Pair Encoding (BPE) étape par étape afin d'apprendre un ensemble de **k=3** nouveaux tokens.
2. Ensuite, l'utiliser afin d'extraire les tokens du **texte** suivant : *used code*

K	Splits	Pairs Frequencies	Frequent Pair / Merges	New Vocab
K = ...	...	...	...	Vocab = { ... }

Pour TP : Vérifier le résultat en exécutant l'algorithme BPE sur machine. Suivre ces étapes :

- Copier le dossier *BPE_Tokenizer* remis. Y accéder.
- Copier le dossier huggingface et le coller dans : C:\Users\USERXYZ\.cache
- Remplacer le contenu de *corpus.txt* dans le dossier BPE_Tokenizer par votre **corpus**.
- Ouvrir le fichier script *test.py*, remplacer par vos valeurs les variables *text* et *vocab_size* comme suit : *text="used code"* et *vocab_size=10*.
- Exécuter le script depuis Anaconda Prompt avec la commande : **> python test.py**
- Vérifier le résultat obtenu avec le vôtre.

Exercice 3

- Soit une chaîne de caractères : *my_str* = "This is a string variable that contains many characters like : A,), Œ". Ecrire une fonction *isNonASCII(str)* qui retourne True si la chaîne envoyée en paramètre contient des caractères non ascii, False si non.
- Encoder la chaîne de caractères *my_str* en ASCII byte. Que remarquez-vous ? Résoudre le problème avec l'approche de votre choix.

Exercice 4

Demander à l'utilisateur d'entrer une chaîne de caractères. Ecrire le code qui permet de récupérer et d'afficher un dictionnaire qui a comme clé le caractère et comme valeur le point de code de ce caractère.

Exercice 5

Ecrire une fonction `my_tokenizer(text)` permettant d'implémenter votre propre tokenizer. La tokenization s'effectue en deux étapes :

- D'abord, en se basant sur les white spaces (espaces blanc).
- Puis, en appliquant la suppression de suffixes pour chaque tokens. Considérés comme suffixes : . , ; ! ? ' ‘
- NB : les mots suivants sont considérés comme tokens : DZ. | goin' | Aug.

Tester votre tokenizer sur la chaine suivante : "I am goin' to travel to DZ. Next Aug. hopefully."

Exercice 6

- Soit le fichier *index.html* contenant le texte suivant :

```
<html lang="fr">
<head>
    <title>Exo de-htmliseur</title>
</head>

<body>
    <h1>Exo de-htmliseur</h1>
    <p>On souhaite retirer toutes les balises de ce fichier.</p>
</body>
</html>
```

- Ecrire un code qui lit le fichier HTML *index.html* et qui renvoie à l'écran tout le texte de ce fichier sans les balises HTML. Utiliser la fonction `sub()` du module `re`.

NB : Pour ouvrir et lire un fichier :

```
f = open("D:\\myfiles\\index.html", "r")
print(f.read())
f.close()
```

La méthode `re.sub()` permet d'effectuer des remplacements assez puissants dans une chaîne de caractères. `re.sub(regex, chaine1, chaine2)` remplace toutes les occurrences de regex trouvées par l'expression régulière dans chaine2 par chaine1.

Exercice 7

- Écrire une fonction *is_palindrome* qui prend comme paramètre une chaîne de caractères et qui retourne un booléen indiquant si la chaîne est un palindrome. Un palindrome est un mot ou une chaîne de caractères qui se lit de la même manière de gauche à droite et de droite à gauche.
- Écrire un programme qui demande à l'utilisateur d'entrer une chaîne de caractères, la tokenise (whitespace) et la transforme en une liste de mots, puis affiche le nombre de mots palindromes présents dans la chaîne.
- Tester le programme avec la chaine : "radar bonjour level python kayak hello civic"